Fundamentos de Mensageria com ActiveMQ e JMS

Este material cobre a transição arquitetural de sistemas fortemente acoplados para ecossistemas distribuídos assíncronos, abordando desde os conceitos fundamentais de desacoplamento até a configuração de brokers tradicionais via Docker.

1. O Problema: O Gargalo da Arquitetura Síncrona (REST/HTTP)

Para entender o valor de um Message Broker, precisamos analisar a falha fundamental do modelo de integração direta (Point-to-Point Integration).

Imagine o portal acadêmico da PUC Goiás no fechamento do semestre. A secretaria envia uma requisição HTTP para a API do "Sistema de Notas", solicitando o processamento das médias de 100 alunos. Em uma **Arquitetura Síncrona**:

1. O Sistema de Notas recebe o *request* e abre uma transação no banco de dados.
2. Em seguida, ele abre uma conexão SMTP para o Servidor de E-mails para enviar as 100 notificações.
3. A *Thread* (linha de execução) do Sistema de Notas fica **bloqueada** (aguardando I/O de rede) até que o Servidor de E-mails responda.

O Ponto de Falha: Se o Servidor de E-mails estiver lento ou sofrer uma queda (Timeout), a requisição original falha. O banco de dados sofre um *Rollback*, a nota não é lançada, e o usuário da secretaria recebe um erro "504 Gateway Timeout". Os sistemas estão **fortemente acoplados temporalmente e espacialmente**.

A Solução: Arquitetura Orientada a Eventos (EDA)

A Mensageria introduz o princípio do **desacoplamento**. A secretaria submete as notas, o Sistema de Notas salva no banco e gera um "Evento" (um pacote de dados dizendo *"Notas da turma X publicadas"*). Este evento é enviado para um intermediário (o Broker). O Sistema de Notas não se importa com quem vai ler esse evento ou quando. Ele simplesmente entrega o pacote e devolve um "HTTP 200 OK" para a secretaria em milissegundos.

---

2. Apache ActiveMQ

O **Apache ActiveMQ** é o nosso Message Broker intermediário. Ele não é apenas um "tubo" por onde a informação passa; ele é um servidor de aplicação complexo projetado para armazenamento e roteamento garantido.

Mecânica Interna e Persistência

O coração da confiabilidade do ActiveMQ é o padrão **Store-and-Forward**. Quando o broker recebe uma mensagem, ele não tenta simplesmente repassá-la. Ele a escreve imediatamente em disco utilizando um mecanismo de armazenamento otimizado chamado **KahaDB**.

Se o servidor do broker sofrer uma queda de energia catastrófica um segundo após receber a mensagem, a informação não será perdida. Ao reiniciar, o ActiveMQ lê o KahaDB e coloca a mensagem de volta na fila.

Conectividade (Transport Connectors)

Diferente das APIs REST que expõem portas HTTP (80/443), o ActiveMQ expõe um conector TCP nativo rodando o protocolo binário **OpenWire** (geralmente na porta **61616**). Este protocolo é altamente otimizado para clientes Java.

---

3. A Especificação JMS

O **JMS (Java Message Service)** não é um framework de terceiros, é uma API padrão da plataforma Java. Ele define as interfaces que os provedores (como a Apache) devem implementar. Para escrever código, seus alunos precisam dominar os seguintes objetos do modelo JMS:

1. **ConnectionFactory:** A fábrica de conexões. É o objeto responsável por encapsular o IP, porta e credenciais do broker, estabelecendo um túnel TCP físico de rede.
2. **Session:** Um contexto *single-threaded* para produzir e consumir mensagens. É na Session que definimos se as mensagens participarão de transações (se uma falhar, todas falham) e como os reconhecimentos de entrega (*Acknowledgments* - ACKs) serão gerenciados.
3. **Destinations (Destinos):** O roteamento lógico dentro do broker.
    - **Queues (Filas):** Implementam o *Point-to-Point Channel Pattern*. A mensagem enviada para uma Queue será consumida por **apenas um** consumidor. Se você tiver 3 instâncias do seu microsserviço rodando, o broker fará o balanceamento de carga (Round-Robin) entre elas.
    - **Topics (Tópicos):** Implementam o padrão *Publish-Subscribe* (Pub/Sub). A mensagem é "bordeada" (broadcast). Todo consumidor que estiver inscrito naquele tópico receberá uma cópia exata da mensagem.
4. **Message:** O pacote em si, que possui três camadas:
    - *Headers:* Metadados do próprio JMS (ex: JMSMessageID, JMSTimestamp).
    - *Properties:* Metadados customizados para filtragem técnica (ex: id_tenant = 5).
    - *Payload (Body):* O conteúdo de negócio.

---

4. Laboratório: Sistema de Notificações Acadêmicas

Nesta prática, vamos conectar uma aplicação Java (Spring Boot) ao broker para simular o envio assíncrono de notificações de notas.

4.1. Isolamento de Infraestrutura (Docker)

Para evitar problemas de variáveis de ambiente no sistema operacional, utilizamos o Docker para hospedar o broker localmente.

O arquivo docker-compose.yml instrui o motor do Docker a criar uma rede isolada, baixar a imagem do ActiveMQ e mapear as portas.

```yaml
version: '3.8'
services:
  activemq:
    image: rmohr/activemq:latest
    container_name: broker-academico
    ports:
      - "61616:61616" # Túnel TCP para o protocolo OpenWire (JMS)
      - "8161:8161"   # Porta HTTP para o Console de Administração Web
```

Comando de execução: docker-compose up -d.

4.2. Configurando o Spring Boot

O Spring Boot facilita o uso do JMS escondendo a complexidade da ConnectionFactory por trás de uma classe chamada JmsTemplate. No entanto, se enviarmos um objeto Java diretamente pelo JmsTemplate, ele usará a serialização nativa do Java (gerando um blob binário ilegível para outras linguagens).

Para garantir que o nosso sistema seja escalável e interoperável (por exemplo, se o *frontend* em React precisar ler uma fila web via STOMP no futuro), configuramos o **Jackson** para transformar todos os objetos Java em **JSON** antes de chegarem ao broker.

Dependência Maven: spring-boot-starter-activemq.

Classe de Configuração:

Java

```java
@Configuration
public class ActiveMQConfig {

  @Bean // Este Bean sobrescreve o conversor padrão do Spring
  public MessageConverter jacksonJmsMessageConverter() {
    MappingJackson2MessageConverter converter = new
MappingJackson2MessageConverter();
    // Força a conversão para Texto (JSON puro) no broker
    converter.setTargetType(MessageType.TEXT);
    // Adiciona um Header na mensagem JMS chamado "_type" com o nome da classe original,
    // permitindo que o consumidor saiba como desserializar o JSON de volta para objeto.
    converter.setTypeIdPropertyName("_type");
    return converter;
  }
}
```

4.3. O Produtor (O Sistema que Emite o Evento)

O Produtor atua como o cliente (Secretaria). Ele encapsula os dados em um objeto de transferência (AvisoDTO) e utiliza o JmsTemplate.

O que acontece nos bastidores: O JmsTemplate solicita uma conexão à ConnectionFactory, cria uma Session, cria um MessageProducer, converte o DTO em JSON, injeta no protocolo OpenWire, envia para a porta 61616, aguarda a confirmação do broker e fecha os recursos. Tudo em uma linha de código.

Java

```java
@Service
public class SecretariaProducer {

  @Autowired
  private JmsTemplate jmsTemplate; // Abstração poderosa do Spring

  public void notificarLancamentoDeNota(AvisoDTO aviso) {
    // Envia o payload para a Destination chamada "queue.notas.academicas"
```

```
        jmsTemplate.convertAndSend("queue.notas.academicas", aviso);
        System.out.println("Transação local concluída. Evento despachado para o Broker.");
    }
}
```

4.4. O Consumidor (O Sistema que Processa)

O consumidor roda de forma totalmente autônoma. Ao utilizarmos a anotação @JmsListener, o Spring instancia um contêiner de mensagens em *background* (*DefaultMessageListenerContainer*).

Esta *thread* abre uma conexão persistente com o ActiveMQ. Quando o broker detecta uma mensagem na Queue, ele a "empurra" (*Push*) para o consumidor. O Spring pega o JSON, lê o *Header* _type, desserializa o dado e injeta no método.

Java

```java
@Component
public class ServicoNotificacaoConsumer {

    // Fica em "escuta" aguardando eventos do broker
    @JmsListener(destination = "queue.notas.academicas")
    public void processarEnvioAssincrono(AvisoDTO aviso) {
        System.out.println("[WORKER] Iniciando processamento pesado em background...");
        System.out.println("Preparando notificação para Matrícula: " + aviso.getMatricula());

        // Aqui o sistema faria o I/O bloqueante (ex: chamada de rede para API SMTP)
        // sem travar a aplicação principal da secretaria.
    }
}
```